Cloudflare AI Dental Assistant

https://gemini.google.com/app/c15dec68d1173d3f

User prompt: Create an ai assistant that we deploy on cloudflare with no cost that can handle answer queries on web chat and handle online bookings for dentist clinic

Response: You can build and deploy a production-ready dental receptionist AI assistant on Cloudflare for **$0/month** by combining the **Cloudflare Agents SDK** (`@cloudflare/ai-chat`), **Workers AI**, **SQLite-backed Durable Objects**, and **D1.**

Zero-Cost Architecture & Free Tier Quotas

Every component in this stack runs entirely within Cloudflare's recurring free tier—no credit card or external cloud database required:

Component	Role in Dental Assistant	Free Tier Allowance
Workers & Static Assets	Hosts the React web chat widget & API routing	**100,000 requests/day**
Durable Objects (SQLite)	Stateful per-patient chat sessions (`AIChatAgent`) & WebSocket streaming	**~3M requests/mo**, **5 GB SQLite storage**
Workers AI	Runs `@cf/meta/llama-3.3-70b-instruct-fp8-fast` with native tool calling	**10,000 Neurons/day** (~250+ chat turns/day)
Cloudflare D1	Shared relational SQL database for clinic-wide appointment slots & bookings	**5M rows read/day**, **100K rows written/day**, **5 GB**

Step-by-Step Implementation

1 **Scaffold Project & Install Dependencies**

Requires Node.js 20+

Initialize a new Cloudflare Worker project and install the Cloudflare Agents SDK, Vercel AI SDK core, and the Workers AI provider:

```
npm create cloudflare@latest dental-ai-assistant -- --type=hello-world --ts --git
cd dental-ai-assistant
npm install agents @cloudflare/ai-chat ai @ai-sdk/react workers-ai-provider zod
```

2 **Provision the D1 Appointments Database**

Run once to create and initialize your SQL database

Create the clinic's shared D1 database to store appointments across all patient sessions:

```
npx wrangler d1 create dental-appointments-db
```

Copy the `database_id` output from the terminal into your `wrangler.jsonc` (shown in the next step), then create `schema.sql`:

```
-- schema.sql
CREATE TABLE IF NOT EXISTS appointments (
  id TEXT PRIMARY KEY,
  patient_name TEXT NOT NULL,
  patient_phone TEXT NOT NULL,
  service_type TEXT NOT NULL,
  appointment_date TEXT NOT NULL, -- YYYY-MM-DD
  appointment_time TEXT NOT NULL, -- HH:MM (e.g., 09:00, 14:00)
  status TEXT NOT NULL DEFAULT 'confirmed',
  created_at TEXT NOT NULL
);

CREATE UNIQUE INDEX IF NOT EXISTS idx_unique_slot
ON appointments(appointment_date, appointment_time)
WHERE status = 'confirmed';
```

Apply the schema locally and to your remote free-tier D1 database:

```
npx wrangler d1 execute dental-appointments-db --local --file=./schema.sql
npx wrangler d1 execute dental-appointments-db --remote --file=./schema.sql
```

3 **Configure wrangler.jsonc**

Includes SQLite Durable Objects migration required by AIChatAgent

Replace your `wrangler.jsonc` with the following configuration, substituting your `database_id`:

```
{
  "$schema": "node_modules/wrangler/config-schema.json",
  "name": "dental-ai-assistant",
  "main": "src/server.ts",
  "compatibility_date": "2026-10-15",
  "compatibility_flags": ["nodejs_compat"],
  "assets": {
    "directory": "./public",
    "binding": "ASSETS"
  },
  "ai": {
    "binding": "AI"
  },
  "d1_databases": [
    {
      "binding": "DB",
      "database_name": "dental-appointments-db",
      "database_id": "YOUR-D1-DATABASE-ID"
    }
  ],
  "durable_objects": {
    "bindings": [
      {
        "name": "DentalAgent",
        "class_name": "DentalAgent"
      }
    ]
  },
  "migrations": [
    {
      "tag": "v1",
      "new_sqlite_classes": ["DentalAgent"]
    }
  ],
  "observability": {
    "enabled": true,
    "traces": { "enabled": true }
  }
}
```

Generate strongly typed bindings (`Cloudflare.Env`) so TypeScript recognizes `env.AI`, `env.DB`, and `env.DentalAgent`:

```
npx wrangler types
```

4 **Build the Stateful Dental Agent Server (src/server.ts)**

Create `src/server.ts`. This extends `AIChatAgent` to persist each visitor's conversation history in its own embedded SQLite Durable Object while querying the shared `env.DB` D1 database to check slots and book appointments without double-booking:

```
import { AIChatAgent, type OnChatMessageOptions } from "@cloudflare/ai-chat";
import { routeAgentRequest } from "agents";
import { streamText, tool, convertToModelMessages, stepCountIs } from "ai";
import { createWorkersAI } from "workers-ai-provider";
import { z } from "zod";

const CLINIC_HOURS = ["09:00", "10:00", "11:00", "13:00", "14:00", "15:00", "16:00"];

const SYSTEM_PROMPT = `You are SmileCare Dental Clinic's friendly virtual receptionist.
Clinic Info:
- Hours: Monday–Saturday, 9:00 AM to 5:00 PM (Closed 12:00 PM–1:00 PM for lunch; Closed Sundays).
- Address: 120 Dental Plaza, Suite 4B.
- Services & Pricing:
  * Routine Checkup & Cleaning: $90 (45 mins)
  * Teeth Whitening: $250 (60 mins)
  * Composite Filling: $150–$280 per tooth
  * Root Canal Consultation: $120
  * Emergency Toothache Exam: $100
- Insurance: We accept Delta Dental, Cigna, MetLife, Aetna, and self-pay.

Booking Rules:
1. Always call checkAvailableSlots before promising a time slot.
2. Before calling bookAppointment, confirm you have the patient's full name, phone number, service type, date (YYYY-
3. Keep answers warm, concise, and helpful. Never diagnose medical conditions; for severe swelling or bleeding, advi
```

```
export class DentalAgent extends AIChatAgent<Env> {
  async onChatMessage(onFinish: unknown, options?: OnChatMessageOptions) {
    const workersai = createWorkersAI({ binding: this.env.AI });

    const result = streamText({
      model: workersai("@cf/meta/llama-3.3-70b-instruct-fp8-fast"),
      system: SYSTEM_PROMPT,
      messages: await convertToModelMessages(this.messages),
      abortSignal: options?.abortSignal,
      stopWhen: stepCountIs(5),
      tools: {
        checkAvailableSlots: tool({
          description: "Check available dental appointment time slots for a specific date (YYYY-MM-DD).",
          inputSchema: z.object({
            date: z.string().describe("Date in YYYY-MM-DD format"),
          }),
          execute: async ({ date }) => {
            const { results } = await this.env.DB.prepare(
              "SELECT appointment_time FROM appointments WHERE appointment_date = ? AND status = 'confirmed'"
            )
              .bind(date)
              .all<{ appointment_time: string }>();

            const booked = new Set(results.map((r) => r.appointment_time));
            const available = CLINIC_HOURS.filter((slot) => !booked.has(slot));

            return {
              date,
              availableSlots: available,
              note: available.length === 0 ? "Fully booked on this date." : "Times are in 24-hour HH:MM format.",
            };
          },
        }),

        bookAppointment: tool({
          description: "Book a dental appointment slot after verifying availability and collecting patient details."
          inputSchema: z.object({
            patientName: z.string().describe("Full name of the patient"),
            patientPhone: z.string().describe("Contact phone number"),
            serviceType: z.string().describe("Dental service requested (e.g., Routine Cleaning, Emergency Exam)"),
            date: z.string().describe("Appointment date in YYYY-MM-DD format"),
            time: z.string().describe("Appointment time in 24-hour HH:MM format from available slots"),
          }),
          execute: async ({ patientName, patientPhone, serviceType, date, time }) => {
            if (!CLINIC_HOURS.includes(time)) {
              return { success: false, error: `Invalid slot. Choose from: ${CLINIC_HOURS.join(", ")}` };
            }

            const id = crypto.randomUUID();
            try {
              await this.env.DB.prepare(
                `INSERT INTO appointments (id, patient_name, patient_phone, service_type, appointment_date, appointm
                 VALUES (?, ?, ?, ?, ?, ?, 'confirmed', ?)`
              )
                .bind(id, patientName, patientPhone, serviceType, date, time, new Date().toISOString())
                .run();

              return {
                success: true,
                confirmationCode: id.slice(0, 8).toUpperCase(),
                summary: `${serviceType} booked for ${patientName} on ${date} at ${time}.`,
              };
            } catch {
              return {
                success: false,
                error: "That time slot was just taken. Please check available slots again.",
              };
            }
          },
        }),

        cancelAppointment: tool({
          description: "Cancel an existing appointment using the patient's phone number and appointment date.",
          inputSchema: z.object({
```

```
            patientPhone: z.string().describe("Patient phone number used during booking"),
            date: z.string().describe("Appointment date in YYYY-MM-DD format"),
          }),
          execute: async ({ patientPhone, date }) => {
            const res = await this.env.DB.prepare(
              "UPDATE appointments SET status = 'cancelled' WHERE patient_phone = ? AND appointment_date = ? AND sta
            )
              .bind(patientPhone, date)
              .run();

            return res.meta.changes > 0
              ? { success: true, message: "Appointment cancelled." }
              : { success: false, message: "No matching confirmed appointment found." };
          },
        }),
      },
    });

    return result.toUIMessageStreamResponse();
  }
}

export default {
  async fetch(request: Request, env: Env): Promise<Response> {
    const agentResponse = await routeAgentRequest(request, env);
    if (agentResponse) return agentResponse;
    return env.ASSETS.fetch(request);
  },
} satisfies ExportedHandler<Env>;
```

5 **Add the Web Chat Widget UI**

Uses @cloudflare/ai-chat/react over WebSockets

If you are bundling a React frontend (e.g., with Vite), connect directly to `DentalAgent` using `useAgent` and `useAgentChat`. Because incoming WebSocket messages on Durable Objects are billed at a 20:1 ratio, real-time streaming barely touches your free-tier request quota:

```
// src/client.tsx
import React, { useState } from "react";
import { useAgent } from "agents/react";
import { useAgentChat } from "@cloudflare/ai-chat/react";

export function DentalChatWidget() {
  // Persist session ID per browser so patients keep their history on refresh
  const [sessionId] = useState(() => {
    const saved = localStorage.getItem("dental_session_id");
    if (saved) return saved;
    const next = crypto.randomUUID();
    localStorage.setItem("dental_session_id", next);
    return next;
  });

  const [input, setInput] = useState("");
  const agent = useAgent({ agent: "DentalAgent", name: sessionId });
  const { messages, sendMessage, status } = useAgentChat({ agent });

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    if (!input.trim() || status === "streaming") return;
    sendMessage({ text: input });
    setInput("");
  };

  return (
    <div className="fixed bottom-4 right-4 w-96 rounded-2xl shadow-xl border bg-white flex flex-col h-[520px] overfl
      <div className="bg-teal-600 text-white px-4 py-3 font-semibold flex justify-between items-center">
        <span>SmileCare Dental Assistant</span>
        <span className="text-xs bg-teal-700 px-2 py-0.5 rounded-full">Online</span>
      </div>

      <div className="flex-1 p-4 overflow-y-auto space-y-3 text-sm">
        {messages.length === 0 && (
          <p className="text-gray-500">
            Hi! Ask me about our dental services, pricing, insurance, or book an appointment online.
          </p>
```

```
          )}
          {messages.map((m) => (
            <div key={m.id} className={m.role === "user" ? "text-right" : "text-left"}>
              <div
                className={`inline-block px-3 py-2 rounded-xl max-w-[85%] ${
                  m.role === "user" ? "bg-teal-600 text-white" : "bg-gray-100 text-gray-800"
                }`}
              >
                {m.parts.map((part, i) =>
                  part.type === "text" ? <span key={i}>{part.text}</span> : null
                )}
              </div>
            </div>
          ))}
        </div>

        <form onSubmit={handleSubmit} className="p-3 border-t flex gap-2">
          <input
            value={input}
            onChange={(e) => setInput(e.target.value)}
            placeholder="Ask a question or book a visit..."
            className="flex-1 border rounded-lg px-3 py-2 text-sm focus:outline-none focus:border-teal-600"
          />
          <button
            type="submit"
            disabled={status === "streaming"}
            className="bg-teal-600 text-white px-4 py-2 rounded-lg text-sm font-medium disabled:opacity-50"
          >
            Send
          </button>
        </form>
      </div>
    );
}
```

6 **Test Locally & Deploy to Cloudflare**

Run the local development server to test chat responses and D1 slot bookings, then deploy globally:

```
# Run locally
npx wrangler dev

# Deploy to *.workers.dev for free
npx wrangler deploy
```

Important Free-Tier Guardrails to Keep It $0

- **Prevent Double-Bookings Automatically:** The `UNIQUE INDEX idx_unique_slot` in D1 guarantees that even if two patients ask the AI for the `10:00` slot at the exact same millisecond, SQLite will reject the second insert and the tool will gracefully ask the patient to pick another time.
- **Maximize Daily Free Neurons:** `@cf/meta/llama-3.3-70b-instruct-fp8-fast` handles multi-step tool calling reliably while staying lightweight. If your clinic traffic grows and approaches the 10,000 Neurons/day cap, switch the model string to `@cf/ibm-granite/granite-4.0-h-micro` or `@cf/meta/llama-3.1-8b-instruct-fast` to stretch the free allowance to 500+ conversations per day.

User prompt: Add a password-protected admin endpoint and UI table to view, filter, and manage all D1 dental appointments.

Response: To add a zero-cost, password-protected admin portal to your existing Worker, store an admin password in **Wrangler Secrets** and verify it on `/api/admin/*` routes using constant-time comparison (`crypto.subtle.timingSafeEqual`).

Here is how to add the backend API, secret configuration, and admin dashboard UI.

1. Set Your Admin Password Secret

Never hardcode your admin password in `wrangler.jsonc` or source code. Use Wrangler Secrets for production, and a `.dev.vars` file for local development:

```
# Set the production secret (prompts you to type your password)
npx wrangler secret put ADMIN_PASSWORD
```

For local testing (`npx wrangler dev`), create a `.dev.vars` file in your project root (make sure `.dev.vars` is in `.gitignore`):

```
# .dev.vars
ADMIN_PASSWORD=super-secret-clinic-pass
```

Then regenerate your Worker types so TypeScript knows `ADMIN_PASSWORD` is on `Env` (or pass `--include-vars`):

```
npx wrangler types
```

2. Add Protected Admin API Routes (`src/server.ts`)

Update the `fetch` handler in `src/server.ts` to expose `/api/admin/appointments` (`GET` to list/filter, `PATCH` to update status or reschedule, and `POST` for manual reception walk-ins) protected by Bearer token verification:

```
// Add to src/server.ts (above or below DentalAgent class)

function verifyAdminPassword(request: Request, env: Env & { ADMIN_PASSWORD?: string }): boolean {
  const authHeader = request.headers.get("Authorization") ?? "";
  const token = authHeader.startsWith("Bearer ") ? authHeader.slice(7) : "";
  const expected = env.ADMIN_PASSWORD ?? "";

  if (!token || !expected) return false;

  const encoder = new TextEncoder();
  const a = encoder.encode(token);
  const b = encoder.encode(expected);

  if (a.byteLength !== b.byteLength) return false;
  return crypto.subtle.timingSafeEqual(a, b);
}

async function handleAdminApi(request: Request, env: Env & { ADMIN_PASSWORD?: string }): Promise<Response> {
  if (!verifyAdminPassword(request, env)) {
    return Response.json({ error: "Unauthorized" }, { status: 401 });
  }

  const url = new URL(request.url);

  // GET /api/admin/appointments?date=YYYY-MM-DD&status=confirmed&search=...
  if (request.method === "GET" && url.pathname === "/api/admin/appointments") {
    const date = url.searchParams.get("date");
    const status = url.searchParams.get("status");
    const search = url.searchParams.get("search");

    let query = "SELECT * FROM appointments WHERE 1=1";
    const params: string[] = [];

    if (date) {
      query += " AND appointment_date = ?";
      params.push(date);
    }
    if (status && status !== "all") {
      query += " AND status = ?";
      params.push(status);
    }
    if (search) {
      query += " AND (patient_name LIKE ? OR patient_phone LIKE ?)";
      params.push(`%${search}%`, `%${search}%`);
    }

    query += " ORDER BY appointment_date ASC, appointment_time ASC LIMIT 250";

    const { results } = await env.DB.prepare(query).bind(...params).all();
    return Response.json({ appointments: results });
  }

  // PATCH /api/admin/appointments/:id (Update status or reschedule)
  const match = url.pathname.match(/^\/api\/admin\/appointments\/([a-zA-Z0-9-]+)$/);
  if (request.method === "PATCH" && match) {
    const id = match[1];
    const body = await request.json<{
      status?: "confirmed" | "completed" | "cancelled" | "no_show";
      appointment_date?: string;
      appointment_time?: string;
    }>();

    if (body.status) {
      await env.DB.prepare("UPDATE appointments SET status = ? WHERE id = ?")
        .bind(body.status, id)
```

```
        .run();
    }

    if (body.appointment_date && body.appointment_time) {
      try {
        await env.DB.prepare(
          "UPDATE appointments SET appointment_date = ?, appointment_time = ? WHERE id = ?"
        )
          .bind(body.appointment_date, body.appointment_time, id)
          .run();
      } catch {
        return Response.json(
          { error: "Target slot is already booked by another confirmed appointment." },
          { status: 409 }
        );
      }
    }

    return Response.json({ success: true });
  }

  // DELETE /api/admin/appointments/:id
  if (request.method === "DELETE" && match) {
    const id = match[1];
    await env.DB.prepare("DELETE FROM appointments WHERE id = ?").bind(id).run();
    return Response.json({ success: true });
  }

  return Response.json({ error: "Not found" }, { status: 404 });
}
```

Then update your default `fetch` export at the bottom of `src/server.ts` so `/api/admin/*` routes are handled before static assets:

```
export default {
  async fetch(request: Request, env: Env): Promise<Response> {
    const url = new URL(request.url);

    // 1. Admin API routes
    if (url.pathname.startsWith("/api/admin/")) {
      return handleAdminApi(request, env);
    }

    // 2. Agents SDK WebSocket & HTTP routes
    const agentResponse = await routeAgentRequest(request, env);
    if (agentResponse) return agentResponse;

    // 3. Static Assets (public/index.html, public/admin.html, etc.)
    return env.ASSETS.fetch(request);
  },
} satisfies ExportedHandler<Env>;
```

3. Build the Dentist Admin Dashboard UI

You can serve the admin dashboard either as a React component or as a standalone zero-build page at `public/admin.html` (accessible at `[https://your-worker.workers.dev/admin.html](https://your-worker.workers.dev/admin.html)`). Below are both options—pick whichever matches your frontend setup.

Option A: Standalone `public/admin.html` (Zero Build Step Required)

Drop this file into `public/admin.html`. Cloudflare Workers Static Assets will automatically serve it at `/admin.html` or `/admin`:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>SmileCare Dental — Admin Bookings</title>
  <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-slate-50 text-slate-800 min-h-screen">
  <!-- Login Modal / Gate -->
  <div id="login-screen" class="min-h-screen flex items-center justify-center p-4">
    <form id="login-form" class="bg-white p-6 rounded-2xl shadow-lg border w-full max-w-sm space-y-4">
      <h1 class="text-xl font-bold text-teal-700">SmileCare Admin Login</h1>
```

```
    <p class="text-xs text-slate-500">Enter your clinic admin password to view and manage appointments.</p>
    <input
      id="password-input"
      type="password"
      placeholder="Admin password"
      required
      class="w-full border rounded-lg px-3 py-2 text-sm focus:outline-none focus:border-teal-600"
    />
    <p id="login-error" class="text-xs text-red-600 hidden">Invalid admin password.</p>
    <button type="submit" class="w-full bg-teal-600 hover:bg-teal-700 text-white font-medium py-2 rounded-lg text-sm">
      Unlock Dashboard
    </button>
  </form>
</div>

<!-- Dashboard View -->
<div id="dashboard-screen" class="hidden max-w-6xl mx-auto p-6 space-y-6">
  <header class="flex flex-wrap justify-between items-center gap-4 border-b pb-4">
    <div>
      <h1 class="text-2xl font-bold text-slate-900">Dental Appointments</h1>
      <p class="text-sm text-slate-500">Real-time bookings from the AI Receptionist (D1 Database)</p>
    </div>
    <div class="flex gap-2">
      <button id="refresh-btn" class="bg-teal-600 text-white px-4 py-2 rounded-lg text-sm font-medium hover:bg-teal-70
        Refresh
      </button>
      <button id="logout-btn" class="border bg-white px-3 py-2 rounded-lg text-sm text-slate-600 hover:bg-slate-100">
        Lock
      </button>
    </div>
  </header>

  <!-- Filters -->
  <div class="bg-white p-4 rounded-xl border shadow-sm flex flex-wrap gap-3 items-end">
    <div>
      <label class="block text-xs font-medium text-slate-500 mb-1">Date</label>
      <input id="filter-date" type="date" class="border rounded-lg px-3 py-1.5 text-sm" />
    </div>
    <div>
      <label class="block text-xs font-medium text-slate-500 mb-1">Status</label>
      <select id="filter-status" class="border rounded-lg px-3 py-1.5 text-sm bg-white">
        <option value="all">All Statuses</option>
        <option value="confirmed">Confirmed</option>
        <option value="completed">Completed</option>
        <option value="cancelled">Cancelled</option>
        <option value="no_show">No-Show</option>
      </select>
    </div>
    <div class="flex-1 min-w-[200px]">
      <label class="block text-xs font-medium text-slate-500 mb-1">Patient Name or Phone</label>
      <input id="filter-search" type="text" placeholder="Search name or phone..." class="w-full border rounded-lg px-3
    </div>
    <button id="clear-filters" class="text-xs text-teal-600 hover:underline px-2 py-2">
      Clear Filters
    </button>
  </div>

  <!-- Table -->
  <div class="bg-white rounded-xl border shadow-sm overflow-x-auto">
    <table class="w-full text-left border-collapse text-sm">
      <thead>
        <tr class="bg-slate-100 text-slate-600 border-b text-xs uppercase">
          <th class="py-3 px-4">Date & Time</th>
          <th class="py-3 px-4">Patient</th>
          <th class="py-3 px-4">Phone</th>
          <th class="py-3 px-4">Service</th>
          <th class="py-3 px-4">Status</th>
          <th class="py-3 px-4 text-right">Actions</th>
        </tr>
      </thead>
      <tbody id="appointments-tbody" class="divide-y"></tbody>
    </table>
  </div>
</div>
```

```
<script>
  let adminToken = sessionStorage.getItem("dental_admin_token") || "";

  const loginScreen = document.getElementById("login-screen");
  const dashboardScreen = document.getElementById("dashboard-screen");
  const loginError = document.getElementById("login-error");
  const tbody = document.getElementById("appointments-tbody");

  const filterDate = document.getElementById("filter-date");
  const filterStatus = document.getElementById("filter-status");
  const filterSearch = document.getElementById("filter-search");

  const badgeColors = {
    confirmed: "bg-teal-100 text-teal-800",
    completed: "bg-blue-100 text-blue-800",
    cancelled: "bg-red-100 text-red-700",
    no_show: "bg-amber-100 text-amber-800",
  };

  async function loadAppointments() {
    const params = new URLSearchParams();
    if (filterDate.value) params.set("date", filterDate.value);
    if (filterStatus.value) params.set("status", filterStatus.value);
    if (filterSearch.value.trim()) params.set("search", filterSearch.value.trim());

    const res = await fetch(`/api/admin/appointments?${params.toString()}`, {
      headers: { Authorization: `Bearer ${adminToken}` },
    });

    if (res.status === 401) {
      sessionStorage.removeItem("dental_admin_token");
      loginScreen.classList.remove("hidden");
      dashboardScreen.classList.add("hidden");
      loginError.classList.remove("hidden");
      return;
    }

    loginScreen.classList.add("hidden");
    dashboardScreen.classList.remove("hidden");
    loginError.classList.add("hidden");

    const data = await res.json();
    renderRows(data.appointments || []);
  }

  function renderRows(appointments) {
    tbody.replaceChildren();
    if (appointments.length === 0) {
      const tr = document.createElement("tr");
      const td = document.createElement("td");
      td.colSpan = 6;
      td.className = "py-8 text-center text-slate-400";
      td.textContent = "No matching appointments found.";
      tr.appendChild(td);
      tbody.appendChild(tr);
      return;
    }

    for (const appt of appointments) {
      const tr = document.createElement("tr");
      tr.className = "hover:bg-slate-50";

      // Date & Time
      const tdDate = document.createElement("td");
      tdDate.className = "py-3 px-4 font-medium whitespace-nowrap";
      tdDate.textContent = `${appt.appointment_date} at ${appt.appointment_time}`;

      // Patient Name
      const tdName = document.createElement("td");
      tdName.className = "py-3 px-4";
      tdName.textContent = appt.patient_name;

      // Phone
```

```
    const tdPhone = document.createElement("td");
    tdPhone.className = "py-3 px-4 text-slate-600";
    tdPhone.textContent = appt.patient_phone;

    // Service
    const tdService = document.createElement("td");
    tdService.className = "py-3 px-4";
    tdService.textContent = appt.service_type;

    // Status Badge
    const tdStatus = document.createElement("td");
    tdStatus.className = "py-3 px-4";
    const span = document.createElement("span");
    span.className = `px-2.5 py-0.5 rounded-full text-xs font-medium ${badgeColors[appt.status] || "bg-slate-100"}`;
    span.textContent = appt.status;
    tdStatus.appendChild(span);

    // Actions
    const tdActions = document.createElement("td");
    tdActions.className = "py-3 px-4 text-right space-x-1 whitespace-nowrap";

    const select = document.createElement("select");
    select.className = "border rounded px-2 py-1 text-xs bg-white";
    ["confirmed", "completed", "cancelled", "no_show"].forEach((st) => {
      const opt = document.createElement("option");
      opt.value = st;
      opt.textContent = st;
      opt.selected = appt.status === st;
      select.appendChild(opt);
    });
    select.addEventListener("change", () => updateStatus(appt.id, select.value));

    const delBtn = document.createElement("button");
    delBtn.className = "text-xs text-red-600 hover:underline ml-2";
    delBtn.textContent = "Delete";
    delBtn.addEventListener("click", () => deleteAppointment(appt.id));

    tdActions.append(select, delBtn);
    tr.append(tdDate, tdName, tdPhone, tdService, tdStatus, tdActions);
    tbody.appendChild(tr);
  }
}

async function updateStatus(id, status) {
  await fetch(`/api/admin/appointments/${id}`, {
    method: "PATCH",
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${adminToken}`,
    },
    body: JSON.stringify({ status }),
  });
  loadAppointments();
}

async function deleteAppointment(id) {
  if (!confirm("Permanently delete this appointment record?")) return;
  await fetch(`/api/admin/appointments/${id}`, {
    method: "DELETE",
    headers: { Authorization: `Bearer ${adminToken}` },
  });
  loadAppointments();
}

document.getElementById("login-form").addEventListener("submit", (e) => {
  e.preventDefault();
  adminToken = document.getElementById("password-input").value;
  sessionStorage.setItem("dental_admin_token", adminToken);
  loadAppointments();
});

document.getElementById("logout-btn").addEventListener("click", () => {
  sessionStorage.removeItem("dental_admin_token");
  adminToken = "";
```

```
      loginScreen.classList.remove("hidden");
      dashboardScreen.classList.add("hidden");
    });

    document.getElementById("refresh-btn").addEventListener("click", loadAppointments);
    filterDate.addEventListener("change", loadAppointments);
    filterStatus.addEventListener("change", loadAppointments);
    filterSearch.addEventListener("input", loadAppointments);

    document.getElementById("clear-filters").addEventListener("click", () => {
      filterDate.value = "";
      filterStatus.value = "all";
      filterSearch.value = "";
      loadAppointments();
    });

    if (adminToken) loadAppointments();
  </script>
</body>
</html>
```

Option B: React Component (`src/AdminDashboard.tsx`)

If you are building a single-page React app alongside `DentalChatWidget`, you can import and render this `<AdminDashboard/>` component on an `/admin` route:

```
// src/AdminDashboard.tsx
import React, { useState, useEffect, useCallback } from "react";

interface Appointment {
  id: string;
  patient_name: string;
  patient_phone: string;
  service_type: string;
  appointment_date: string;
  appointment_time: string;
  status: "confirmed" | "completed" | "cancelled" | "no_show";
}

export function AdminDashboard() {
  const [token, setToken] = useState(() => sessionStorage.getItem("dental_admin_token") || "");
  const [passwordInput, setPasswordInput] = useState("");
  const [isUnlocked, setIsUnlocked] = useState(false);
  const [error, setError] = useState("");

  const [appointments, setAppointments] = useState<Appointment[]>([]);
  const [dateFilter, setDateFilter] = useState("");
  const [statusFilter, setStatusFilter] = useState("all");
  const [search, setSearch] = useState("");

  const fetchAppointments = useCallback(async (activeToken: string) => {
    if (!activeToken) return;
    const params = new URLSearchParams();
    if (dateFilter) params.set("date", dateFilter);
    if (statusFilter !== "all") params.set("status", statusFilter);
    if (search.trim()) params.set("search", search.trim());

    const res = await fetch(`/api/admin/appointments?${params.toString()}`, {
      headers: { Authorization: `Bearer ${activeToken}` },
    });

    if (res.status === 401) {
      setIsUnlocked(false);
      setError("Invalid admin password");
      sessionStorage.removeItem("dental_admin_token");
      return;
    }

    const data = await res.json<{ appointments: Appointment[] }>();
    setAppointments(data.appointments);
    setIsUnlocked(true);
    setError("");
  }, [dateFilter, statusFilter, search]);
```

```
useEffect(() => {
  if (token) fetchAppointments(token);
}, [token, fetchAppointments]);

const handleLogin = (e: React.FormEvent) => {
  e.preventDefault();
  sessionStorage.setItem("dental_admin_token", passwordInput);
  setToken(passwordInput);
};

const updateStatus = async (id: string, status: Appointment["status"]) => {
  await fetch(`/api/admin/appointments/${id}`, {
    method: "PATCH",
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${token}`,
    },
    body: JSON.stringify({ status }),
  });
  fetchAppointments(token);
};

if (!isUnlocked) {
  return (
    <div className="min-h-screen flex items-center justify-center bg-slate-50 p-4">
      <form onSubmit={handleLogin} className="bg-white p-6 rounded-2xl shadow border w-full max-w-sm space-y-4">
        <h2 className="text-xl font-bold text-teal-700">SmileCare Admin</h2>
        <input
          type="password"
          value={passwordInput}
          onChange={(e) => setPasswordInput(e.target.value)}
          placeholder="Enter ADMIN_PASSWORD"
          className="w-full border rounded-lg px-3 py-2 text-sm"
        />
        {error && <p className="text-xs text-red-600">{error}</p>}
        <button type="submit" className="w-full bg-teal-600 text-white py-2 rounded-lg text-sm font-medium">
          Unlock
        </button>
      </form>
    </div>
  );
}

return (
  <div className="max-w-6xl mx-auto p-6 space-y-6">
    <div className="flex justify-between items-center border-b pb-4">
      <h1 className="text-2xl font-bold">Dental Appointments ({appointments.length})</h1>
      <button
        onClick={() => fetchAppointments(token)}
        className="bg-teal-600 text-white px-4 py-2 rounded-lg text-sm font-medium"
      >
        Refresh
      </button>
    </div>

    <div className="flex flex-wrap gap-3 bg-white p-4 rounded-xl border">
      <input
        type="date"
        value={dateFilter}
        onChange={(e) => setDateFilter(e.target.value)}
        className="border rounded-lg px-3 py-1.5 text-sm"
      />
      <select
        value={statusFilter}
        onChange={(e) => setStatusFilter(e.target.value)}
        className="border rounded-lg px-3 py-1.5 text-sm bg-white"
      >
        <option value="all">All Statuses</option>
        <option value="confirmed">Confirmed</option>
        <option value="completed">Completed</option>
        <option value="cancelled">Cancelled</option>
        <option value="no_show">No-Show</option>
      </select>
      <input
```

```
          type="text"
          value={search}
          onChange={(e) => setSearch(e.target.value)}
          placeholder="Search patient name or phone..."
          className="flex-1 border rounded-lg px-3 py-1.5 text-sm"
        />
      </div>

      <div className="bg-white rounded-xl border overflow-x-auto">
        <table className="w-full text-left text-sm">
          <thead className="bg-slate-100 text-xs uppercase text-slate-600 border-b">
            <tr>
              <th className="py-3 px-4">Date & Time</th>
              <th className="py-3 px-4">Patient</th>
              <th className="py-3 px-4">Phone</th>
              <th className="py-3 px-4">Service</th>
              <th className="py-3 px-4">Status</th>
            </tr>
          </thead>
          <tbody className="divide-y">
            {appointments.map((a) => (
              <tr key={a.id} className="hover:bg-slate-50">
                <td className="py-3 px-4 font-medium">{a.appointment_date} {a.appointment_time}</td>
                <td className="py-3 px-4">{a.patient_name}</td>
                <td className="py-3 px-4">{a.patient_phone}</td>
                <td className="py-3 px-4">{a.service_type}</td>
                <td className="py-3 px-4">
                  <select
                    value={a.status}
                    onChange={(e) => updateStatus(a.id, e.target.value as Appointment["status"])}
                    className="border rounded px-2 py-1 text-xs bg-white"
                  >
                    <option value="confirmed">confirmed</option>
                    <option value="completed">completed</option>
                    <option value="cancelled">cancelled</option>
                    <option value="no_show">no_show</option>
                  </select>
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    </div>
  );
}
```

How Cancellations Automatically Free Up Slots for the AI

Because your `idx_unique_slot` index in D1 uses a partial `WHERE status = 'confirmed'` clause (`CREATE UNIQUE INDEX idx_unique_slot ON appointments(appointment_date, appointment_time) WHERE status = 'confirmed'`), changing an appointment's status to `cancelled` or `no_show` in the admin table immediately releases that time slot so the AI assistant can offer it to the next patient.

User prompt: message on telegram email notifications when a patient books

Response: You can send instant **Telegram** and **Email** booking notifications for **$0/month** using standard `fetch()` calls inside your Worker:

- **Telegram Bot API**: 100% free with unlimited messages. You can even include inline Confirm / Cancel buttons right inside the Telegram message so the clinic receptionist can manage bookings from their phone.
- **Resend (Email API):** Free tier includes **3,000 emails/month** (100/day) via a simple REST API call—no heavy Node packages needed.

By wrapping the notification calls in `this.ctx.waitUntil(...)`, the AI assistant responds to the patient immediately while Cloudflare finishes delivering the Telegram message and email in the background.

1. Update Database Schema for Patient Email (Optional)

If you want to send a confirmation email to the patient (in addition to alerting the clinic), add `patient_email` to your D1 table:

```
npx wrangler d1 execute dental-appointments-db --local --command="ALTER TABLE appointments ADD COLUMN patient_email TEXT
npx wrangler d1 execute dental-appointments-db --remote --command="ALTER TABLE appointments ADD COLUMN patient_email TEX
```

2. Configure Your Free API Secrets

1. **Telegram:** Message `@BotFather` on Telegram and run `/newbot` to get your `TELEGRAM_BOT_TOKEN`. Then message `@userinfobot` (or add the bot to your clinic's staff group chat) to get your `TELEGRAM_CHAT_ID`.
2. **Resend:** Create a free account at resend.com and generate a `RESEND_API_KEY`.

Save these credentials securely in **Wrangler Secrets** for production:

```
npx wrangler secret put TELEGRAM_BOT_TOKEN
npx wrangler secret put TELEGRAM_CHAT_ID
npx wrangler secret put RESEND_API_KEY
npx wrangler secret put CLINIC_NOTIFICATION_EMAIL
```

And add them to your `.dev.vars` file for local testing:

```
# .dev.vars
ADMIN_PASSWORD=super-secret-clinic-pass
TELEGRAM_BOT_TOKEN=123456789:ABCdefGHIjklMNOpqrsTUVwxyz
TELEGRAM_CHAT_ID=987654321
RESEND_API_KEY=re_123456789
CLINIC_NOTIFICATION_EMAIL=reception@smilecaredental.com
```

Regenerate your TypeScript types so `Env` includes the secret bindings:

```
npx wrangler types
```

3. Add the Notification Helper (`src/notifications.ts`)

Create `src/notifications.ts` to dispatch both the Telegram message and the Resend email in parallel using `Promise.allSettled`:

```
// src/notifications.ts
export interface BookingNotificationPayload {
  id: string;
  confirmationCode: string;
  patientName: string;
  patientPhone: string;
  patientEmail?: string;
  serviceType: string;
  date: string;
  time: string;
}

export interface NotificationEnv {
  TELEGRAM_BOT_TOKEN?: string;
  TELEGRAM_CHAT_ID?: string;
  RESEND_API_KEY?: string;
  CLINIC_NOTIFICATION_EMAIL?: string;
}

export async function sendBookingNotifications(
  env: NotificationEnv,
  booking: BookingNotificationPayload
): Promise<void> {
  const tasks: Promise<unknown>[] = [];

  // 1. Send Telegram Alert to Clinic Staff / Reception Group
  if (env.TELEGRAM_BOT_TOKEN && env.TELEGRAM_CHAT_ID) {
    const telegramText = [
      `🦷 *New Dental Appointment Booked!*`,
      ``,
      `*Code:* \`${booking.confirmationCode}\``,
      `*Patient:* ${escapeMarkdown(booking.patientName)}`,
      `*Phone:* ${escapeMarkdown(booking.patientPhone)}`,
      booking.patientEmail ? `*Email:* ${escapeMarkdown(booking.patientEmail)}` : null,
      `*Service:* ${escapeMarkdown(booking.serviceType)}`,
      `*Date:* ${booking.date}`,
      `*Time:* ${booking.time}`,
    ]
      .filter(Boolean)
      .join("\n");

    tasks.push(
      fetch(`https://api.telegram.org/bot${env.TELEGRAM_BOT_TOKEN}/sendMessage`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
```

```
          chat_id: env.TELEGRAM_CHAT_ID,
          text: telegramText,
          parse_mode: "Markdown",
          reply_markup: {
            inline_keyboard: [
              [
                { text: "✅ Mark Completed", callback_data: `status:${booking.id}:completed` },
                { text: "❌ Cancel Slot", callback_data: `status:${booking.id}:cancelled` },
              ],
            ],
          },
        }),
      })
    );
  }

  // 2. Send Email Notification (Clinic + Patient if email provided)
  if (env.RESEND_API_KEY && env.CLINIC_NOTIFICATION_EMAIL) {
    const recipients = [env.CLINIC_NOTIFICATION_EMAIL];
    if (booking.patientEmail) {
      recipients.push(booking.patientEmail);
    }

    tasks.push(
      fetch("https://api.resend.com/emails", {
        method: "POST",
        headers: {
          Authorization: `Bearer ${env.RESEND_API_KEY}`,
          "Content-Type": "application/json",
        },
        body: JSON.stringify({
          // Use onboarding@resend.dev for free testing before verifying your custom domain
          from: "SmileCare Dental <onboarding@resend.dev>",
          to: recipients,
          subject: `Appointment Confirmed: ${booking.serviceType} on ${booking.date} at${booking.time}`,
          html: `
            <div style="font-family: sans-serif; max-width: 520px; margin: 0 auto; border: 1px solid #e2e8f0; border-rad
              <h2 style="color: #0d9488; margin-top: 0;">SmileCare Dental Confirmation</h2>
              <p>A new dental appointment has been scheduled:</p>
              <table style="width: 100%; border-collapse: collapse; font-size: 14px;">
                <tr><td style="padding: 6px 0; color: #64748b;">Confirmation Code:</td><td><strong>${booking.confirmatio
                <tr><td style="padding: 6px 0; color: #64748b;">Patient Name:</td><td><strong>${booking.patientName}</st
                <tr><td style="padding: 6px 0; color: #64748b;">Phone:</td><td>${booking.patientPhone}</td></tr>
                <tr><td style="padding: 6px 0; color: #64748b;">Service:</td><td>${booking.serviceType}</td></tr>
                <tr><td style="padding: 6px 0; color: #64748b;">Date & Time:</td><td><strong>${booking.date} at${booking
              </table>
              <p style="font-size: 12px; color: #94a3b8; margin-top: 20px;">
                120 Dental Plaza, Suite 4B • Need to reschedule? Reply to this email or use our web chat.
              </p>
            </div>
          `,
        }),
      })
    );
  }

  await Promise.allSettled(tasks);
}

function escapeMarkdown(text: string): string {
  return text.replace(/[_*[\]()~`>#+=|{}.!-]/g, "\\$&");
}
```

4. Wire Notifications into `bookAppointment` (`src/server.ts`)

Import `sendBookingNotifications` in `src/server.ts` and call it inside `this.ctx.waitUntil(...)` right after inserting the booking into D1:

```
// Inside src/server.ts — update the bookAppointment tool inside DentalAgent:
import { sendBookingNotifications, type NotificationEnv } from "./notifications";

// ... inside tools: { ...
bookAppointment: tool({
  description: "Book a dental appointment slot after verifying availability and collecting patient details.",
  inputSchema: z.object({
```

```
    patientName: z.string().describe("Full name of the patient"),
    patientPhone: z.string().describe("Contact phone number"),
    patientEmail: z.string().email().optional().describe("Optional patient email address for confirmation"),
    serviceType: z.string().describe("Dental service requested"),
    date: z.string().describe("Appointment date in YYYY-MM-DD format"),
    time: z.string().describe("Appointment time in 24-hour HH:MM format from available slots"),
  }),
  execute: async ({ patientName, patientPhone, patientEmail, serviceType, date, time }) => {
    if (!CLINIC_HOURS.includes(time)) {
      return { success: false, error: `Invalid slot. Choose from: ${CLINIC_HOURS.join(", ")}` };
    }

    const id = crypto.randomUUID();
    const confirmationCode = id.slice(0, 8).toUpperCase();

    try {
      await this.env.DB.prepare(
        `INSERT INTO appointments (id, patient_name, patient_phone, patient_email, service_type, appointment_date, appoi
         VALUES (?, ?, ?, ?, ?, ?, ?, 'confirmed', ?)`
      )
        .bind(id, patientName, patientPhone, patientEmail ?? null, serviceType, date, time, new Date().toISOString())
        .run();

      // Dispatch Telegram + Email notifications in the background without slowing down chat response
      this.ctx.waitUntil(
        sendBookingNotifications(this.env as Env & NotificationEnv, {
          id,
          confirmationCode,
          patientName,
          patientPhone,
          patientEmail,
          serviceType,
          date,
          time,
        })
      );

      return {
        success: true,
        confirmationCode,
        summary: `${serviceType} booked for ${patientName} on ${date} at ${time}.`,
      };
    } catch {
      return {
        success: false,
        error: "That time slot was just taken. Please check available slots again.",
      };
    }
  },
}),
```

5. Bonus: Let Staff Cancel or Complete Bookings Directly from Telegram

Notice the inline ✅ **Mark Completed** and ❌ **Cancel Slot** buttons included in `sendBookingNotifications` above. You can add a tiny `/api/telegram/webhook` endpoint to `src/server.ts` so clicking those buttons inside Telegram updates D1 immediately:

```
// Add to src/server.ts
async function handleTelegramWebhook(request: Request, env: Env & NotificationEnv): Promise<Response> {
  const update = await request.json<{
    callback_query?: {
      id: string;
      data?: string;
      message?: { chat: { id: number }; message_id: number; text?: string };
    };
  }>();

  const cb = update.callback_query;
  if (cb?.data?.startsWith("status:")) {
    const [, appointmentId, newStatus] = cb.data.split(":");

    await env.DB.prepare("UPDATE appointments SET status = ? WHERE id = ?")
      .bind(newStatus, appointmentId)
      .run();
```

```
    // Acknowledge button tap in Telegram
    await fetch(`https://api.telegram.org/bot${env.TELEGRAM_BOT_TOKEN}/answerCallbackQuery`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        callback_query_id: cb.id,
        text: `Appointment marked as ${newStatus}!`,
      }),
    });
  }

  return Response.json({ ok: true });
}
```

Register that route in your default `fetch` handler:

```
if (url.pathname === "/api/telegram/webhook" && request.method === "POST") {
  return handleTelegramWebhook(request, env);
}
```

Once deployed (`npx wrangler deploy`), link your Telegram bot to your Worker webhook with a single browser visit or `curl` command:

```
curl "https://api.telegram.org/bot<YOUR_TELEGRAM_BOT_TOKEN>/setWebhook?url=https://dental-ai-assistant.<YOUR_SUBDOMAIN>.
```

User prompt: Show me how to add a free Cloudflare Cron Trigger that queries D1 daily and sends 24-hour appointment reminders via Email and Telegram.

Response: You can automate daily 24-hour appointment reminders for **$0/month** by adding a **Cloudflare Cron Trigger** (`triggers.crons` in `wrangler.jsonc`) and implementing the `scheduled()` handler in your Worker.

Cloudflare Cron Triggers are included on the Workers Free plan (up to 5 cron schedules per account) and run automatically on Cloudflare's edge network—even when nobody is visiting your website.

1. Track Sent Reminders in D1 (Prevent Duplicate Alerts)

Add a `reminder_sent` column to your `appointments` table so the cron job never sends duplicate emails or Telegram messages if it is triggered more than once:

```
npx wrangler d1 execute dental-appointments-db --local --command="ALTER TABLE appointments ADD COLUMN reminder_sent INTE
npx wrangler d1 execute dental-appointments-db --remote --command="ALTER TABLE appointments ADD COLUMN reminder_sent INT
```

2. Add the Cron Schedule to `wrangler.jsonc`

Add a `triggers` block to your `wrangler.jsonc`. Cron expressions on Cloudflare run in **UTC**—for example, `0 0 * * *` runs every day at **8:00 AM Philippine Time (UTC+8)**:

```
{
  "$schema": "node_modules/wrangler/config-schema.json",
  "name": "dental-ai-assistant",
  "main": "src/server.ts",
  "compatibility_date": "2026-10-15",
  "compatibility_flags": ["nodejs_compat"],
  // Runs daily at 00:00 UTC (8:00 AM UTC+8)
  "triggers": {
    "crons": ["0 0 * * *"]
  },
  "assets": {
    "directory": "./public",
    "binding": "ASSETS"
  },
  "ai": {
    "binding": "AI"
  },
  "d1_databases": [
    {
      "binding": "DB",
      "database_name": "dental-appointments-db",
      "database_id": "YOUR-D1-DATABASE-ID"
    }
  ],
  "durable_objects": {
    "bindings": [
      {
```

```
        "name": "DentalAgent",
        "class_name": "DentalAgent"
      }
    ]
  },
  "migrations": [
    {
      "tag": "v1",
      "new_sqlite_classes": ["DentalAgent"]
    }
  ],
  "observability": {
    "enabled": true,
    "traces": { "enabled": true }
  }
}
```

3. Add the Reminder Helper (`src/notifications.ts`)

Append `sendDailyReminders` to `src/notifications.ts`. It sends:

1. **A single morning digest on Telegram** to the clinic staff listing all of tomorrow's confirmed patients.
2. **Individual 24-hour reminder emails via Resend** to each patient who provided an email address.

```
// Append to src/notifications.ts

export interface UpcomingAppointmentRow {
  id: string;
  patient_name: string;
  patient_phone: string;
  patient_email: string | null;
  service_type: string;
  appointment_date: string;
  appointment_time: string;
}

export async function sendDailyReminders(
  db: D1Database,
  env: NotificationEnv,
  tomorrowDate: string
): Promise<{ totalFound: number; emailsSent: number; telegramSent: boolean }> {
  // 1. Fetch tomorrow's confirmed appointments that haven't received a reminder yet
  const { results } = await db
    .prepare(
      `SELECT id, patient_name, patient_phone, patient_email, service_type, appointment_date, appointment_time
       FROM appointments
       WHERE appointment_date = ? AND status = 'confirmed' AND reminder_sent = 0
       ORDER BY appointment_time ASC`
    )
    .bind(tomorrowDate)
    .all<UpcomingAppointmentRow>();

  if (results.length === 0) {
    return { totalFound: 0, emailsSent: 0, telegramSent: false };
  }

  const tasks: Promise<unknown>[] = [];
  let telegramSent = false;
  let emailsSent = 0;

  // 2. Send a single Telegram Daily Digest to the Clinic Staff
  if (env.TELEGRAM_BOT_TOKEN && env.TELEGRAM_CHAT_ID) {
    const lines = results.map(
      (a, idx) =>
        `${idx + 1}. *${a.appointment_time}* — ${escapeMarkdown(a.patient_name)} (${escapeMarkdown(a.service_type)}) • \
    );

    const digestText = [
      `🗓️ *24-Hour Schedule Digest for Tomorrow (${tomorrowDate})*`,
      `Total confirmed appointments: *${results.length}*`,
      ``,
      ...lines,
    ].join("\n");
```

```
    tasks.push(
      fetch(`https://api.telegram.org/bot${env.TELEGRAM_BOT_TOKEN}/sendMessage`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          chat_id: env.TELEGRAM_CHAT_ID,
          text: digestText,
          parse_mode: "Markdown",
        }),
      })
    );
    telegramSent = true;
  }

  // 3. Send Individual 24-Hour Reminder Emails to Patients via Resend Batch API
  if (env.RESEND_API_KEY) {
    const emailPayloads = results
      .filter((a) => Boolean(a.patient_email))
      .map((a) => ({
        from: "SmileCare Dental <onboarding@resend.dev>",
        to: [a.patient_email!],
        subject: `Reminder: Your Dental Appointment Tomorrow (${a.appointment_date} at ${a.appointment_time})`,
        html: `
          <div style="font-family: sans-serif; max-width: 520px; margin: 0 auto; border: 1px solid #e2e8f0; border-radiu
            <h2 style="color: #0d9488; margin-top: 0;">Friendly 24-Hour Appointment Reminder</h2>
            <p>Hi <strong>${a.patient_name}</strong>,</p>
            <p>This is a friendly reminder of your upcoming dental visit tomorrow at <strong>SmileCare Dental Clinic</st
            <ul style="line-height: 1.8; color: #334155;">
              <li><strong>Service:</strong> ${a.service_type}</li>
              <li><strong>Date:</strong> ${a.appointment_date}</li>
              <li><strong>Time:</strong> ${a.appointment_time}</li>
              <li><strong>Location:</strong> 120 Dental Plaza, Suite 4B</li>
            </ul>
            <p style="font-size: 13px; color: #64748b;">
              Please arrive 10 minutes early. Need to reschedule or cancel? Use our online clinic chat assistant anytime
            </p>
          </div>
        `,
      }));

    if (emailPayloads.length > 0) {
      // Resend's /emails/batch endpoint sends up to 100 emails in a single subrequest
      tasks.push(
        fetch("https://api.resend.com/emails/batch", {
          method: "POST",
          headers: {
            Authorization: `Bearer ${env.RESEND_API_KEY}`,
            "Content-Type": "application/json",
          },
          body: JSON.stringify(emailPayloads),
        })
      );
      emailsSent = emailPayloads.length;
    }
  }

  await Promise.allSettled(tasks);

  // 4. Mark all processed appointments as reminder_sent = 1 in a single D1 batch
  const updateStatements = results.map((a) =>
    db.prepare("UPDATE appointments SET reminder_sent = 1 WHERE id = ?").bind(a.id)
  );
  await db.batch(updateStatements);

  return { totalFound: results.length, emailsSent, telegramSent };
}
```

4. Register the `scheduled()` Handler in `src/server.ts`

Update the default export in `src/server.ts` to include the `scheduled` method alongside `fetch`. Make sure to wrap the async task in `ctx.waitUntil(...)` so the Worker stays alive until all D1 updates and API calls finish:

```
// In src/server.ts — import sendDailyReminders
import { sendBookingNotifications, sendDailyReminders, type NotificationEnv } from "./notifications";

// Helper to get tomorrow's YYYY-MM-DD in your clinic's timezone (e.g., Asia/Manila)
function getTomorrowDateString(timeZone = "Asia/Manila"): string {
  const tomorrow = new Date(Date.now() + 24 * 60 * 60 * 1000);
  // 'en-CA' locale formats dates strictly as YYYY-MM-DD
  return new Intl.DateTimeFormat("en-CA", {
    timeZone,
    year: "numeric",
    month: "2-digit",
    day: "2-digit",
  }).format(tomorrow);
}

export default {
  async fetch(request: Request, env: Env): Promise<Response> {
    const url = new URL(request.url);

    if (url.pathname.startsWith("/api/admin/")) {
      return handleAdminApi(request, env);
    }

    if (url.pathname === "/api/telegram/webhook" && request.method === "POST") {
      return handleTelegramWebhook(request, env as Env & NotificationEnv);
    }

    const agentResponse = await routeAgentRequest(request, env);
    if (agentResponse) return agentResponse;

    return env.ASSETS.fetch(request);
  },

  // Runs automatically on the cron schedule defined in wrangler.jsonc
  async scheduled(controller: ScheduledController, env: Env, ctx: ExecutionContext): Promise<void> {
    const tomorrowDate = getTomorrowDateString("Asia/Manila");

    ctx.waitUntil(
      sendDailyReminders(env.DB, env as Env & NotificationEnv, tomorrowDate).then((summary) => {
        console.log(
          `[Cron ${controller.cron}] Processed reminders for ${tomorrowDate}:`,
          JSON.stringify(summary)
        );
      })
    );
  },
} satisfies ExportedHandler<Env>;
```

5. Test the Cron Trigger Locally Before Deploying

You don't have to wait 24 hours to verify that your cron trigger works. Start Wrangler with `--test-scheduled` and trigger the cron event immediately with `curl`:

```
# 1. Start local dev server with scheduled testing enabled
npx wrangler dev --test-scheduled

# 2. In another terminal, fire the scheduled handler manually
curl "http://localhost:8787/__scheduled?cron=0+0+*+*+*"

# 3. Deploy to Cloudflare
npx wrangler deploy
```

> **Why Resend's `/emails/batch` endpoint matters on Cloudflare Free:** Free-tier Workers allow up to **50 outbound subrequests per invocation.** By using `[https://api.resend.com/emails/batch](https://api.resend.com/emails/batch)` and `db.batch()`, sending reminders to 40 patients only uses **1 Resend HTTP request**, **1 Telegram HTTP request**, and **2 D1 queries**—staying well underneath Cloudflare's free runtime limits.

User prompt: lets build it first

Response: Here is the **complete, ready-to-run project** with every file consolidated so you can build, test, and deploy it in a few minutes without needing any frontend build step (no Vite/Webpack config required).

Step 1: Create the Project Directory & Files

Run these commands in your terminal to set up a clean folder and install the dependencies:

```
mkdir dental-ai-assistant && cd dental-ai-assistant
npm init -y
npm install agents @cloudflare/ai-chat ai workers-ai-provider zod
npm install -D wrangler typescript @cloudflare/workers-types
mkdir -p src public
```

Now create these **6 files** in your project:

1. `tsconfig.json`

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ES2022",
    "moduleResolution": "Bundler",
    "strict": true,
    "skipLibCheck": true,
    "lib": ["ES2022"],
    "types": ["@cloudflare/workers-types"]
  },
  "include": ["src/**/*.ts", "worker-configuration.d.ts"]
}
```

2. `schema.sql`

```
CREATE TABLE IF NOT EXISTS appointments (
  id TEXT PRIMARY KEY,
  patient_name TEXT NOT NULL,
  patient_phone TEXT NOT NULL,
  patient_email TEXT,
  service_type TEXT NOT NULL,
  appointment_date TEXT NOT NULL,
  appointment_time TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'confirmed',
  reminder_sent INTEGER NOT NULL DEFAULT 0,
  created_at TEXT NOT NULL
);

CREATE UNIQUE INDEX IF NOT EXISTS idx_unique_slot
ON appointments(appointment_date, appointment_time)
WHERE status = 'confirmed';
```

3. `wrangler.jsonc`

(You will paste your `database_id` into this file in Step 2)

```
{
  "$schema": "node_modules/wrangler/config-schema.json",
  "name": "dental-ai-assistant",
  "main": "src/server.ts",
  "compatibility_date": "2026-10-15",
  "compatibility_flags": ["nodejs_compat"],
  "triggers": {
    "crons": ["0 0 * * *"]
  },
  "assets": {
    "directory": "./public",
    "binding": "ASSETS"
  },
  "ai": {
    "binding": "AI"
  },
  "d1_databases": [
    {
      "binding": "DB",
      "database_name": "dental-appointments-db",
      "database_id": "REPLACE_WITH_YOUR_D1_DATABASE_ID"
    }
  ],
  "durable_objects": {
```

```
    "bindings": [
      {
        "name": "DentalAgent",
        "class_name": "DentalAgent"
      }
    ]
  },
  "migrations": [
    {
      "tag": "v1",
      "new_sqlite_classes": ["DentalAgent"]
    }
  ],
  "observability": {
    "enabled": true,
    "traces": { "enabled": true }
  }
}
```

4. `src/notifications.ts`

```
export interface BookingNotificationPayload {
  id: string;
  confirmationCode: string;
  patientName: string;
  patientPhone: string;
  patientEmail?: string;
  serviceType: string;
  date: string;
  time: string;
}

export interface NotificationEnv {
  TELEGRAM_BOT_TOKEN?: string;
  TELEGRAM_CHAT_ID?: string;
  RESEND_API_KEY?: string;
  CLINIC_NOTIFICATION_EMAIL?: string;
}

export interface UpcomingAppointmentRow {
  id: string;
  patient_name: string;
  patient_phone: string;
  patient_email: string | null;
  service_type: string;
  appointment_date: string;
  appointment_time: string;
}

function escapeMarkdown(text: string): string {
  return text.replace(/[_*[\]()~`>#+=|{}.!-]/g, "\\$&");
}

export async function sendBookingNotifications(
  env: NotificationEnv,
  booking: BookingNotificationPayload
): Promise<void> {
  const tasks: Promise<unknown>[] = [];

  if (env.TELEGRAM_BOT_TOKEN && env.TELEGRAM_CHAT_ID) {
    const telegramText = [
      `🦷 *New Dental Appointment Booked!*`,
      ``,
      `*Code:* \`${booking.confirmationCode}\``,
      `*Patient:* ${escapeMarkdown(booking.patientName)}`,
      `*Phone:* ${escapeMarkdown(booking.patientPhone)}`,
      booking.patientEmail ? `*Email:* ${escapeMarkdown(booking.patientEmail)}` : null,
      `*Service:* ${escapeMarkdown(booking.serviceType)}`,
      `*Date:* ${booking.date}`,
      `*Time:* ${booking.time}`,
    ]
      .filter(Boolean)
      .join("\n");
```

```
    tasks.push(
      fetch(`https://api.telegram.org/bot${env.TELEGRAM_BOT_TOKEN}/sendMessage`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          chat_id: env.TELEGRAM_CHAT_ID,
          text: telegramText,
          parse_mode: "Markdown",
          reply_markup: {
            inline_keyboard: [
              [
                { text: "✅ Mark Completed", callback_data: `status:${booking.id}:completed` },
                { text: "❌ Cancel Slot", callback_data: `status:${booking.id}:cancelled` },
              ],
            ],
          },
        }),
      })
    );
  }

  if (env.RESEND_API_KEY && env.CLINIC_NOTIFICATION_EMAIL) {
    const recipients = [env.CLINIC_NOTIFICATION_EMAIL];
    if (booking.patientEmail) recipients.push(booking.patientEmail);

    tasks.push(
      fetch("https://api.resend.com/emails", {
        method: "POST",
        headers: {
          Authorization: `Bearer ${env.RESEND_API_KEY}`,
          "Content-Type": "application/json",
        },
        body: JSON.stringify({
          from: "SmileCare Dental <onboarding@resend.dev>",
          to: recipients,
          subject: `Appointment Confirmed: ${booking.serviceType} on ${booking.date} at${booking.time}`,
          html: `
            <div style="font-family: sans-serif; max-width: 520px; margin: 0 auto; border: 1px solid #e2e8f0; border-rad
              <h2 style="color: #0d9488; margin-top: 0;">SmileCare Dental Confirmation</h2>
              <p>A new dental appointment has been scheduled:</p>
              <table style="width: 100%; border-collapse: collapse; font-size: 14px;">
                <tr><td style="padding: 6px 0; color: #64748b;">Confirmation Code:</td><td><strong>${booking.confirmatio
                <tr><td style="padding: 6px 0; color: #64748b;">Patient Name:</td><td><strong>${booking.patientName}</st
                <tr><td style="padding: 6px 0; color: #64748b;">Phone:</td><td>${booking.patientPhone}</td></tr>
                <tr><td style="padding: 6px 0; color: #64748b;">Service:</td><td>${booking.serviceType}</td></tr>
                <tr><td style="padding: 6px 0; color: #64748b;">Date & Time:</td><td><strong>${booking.date} at${booking
              </table>
            </div>
          `,
        }),
      })
    );
  }

  await Promise.allSettled(tasks);
}

export async function sendDailyReminders(
  db: D1Database,
  env: NotificationEnv,
  tomorrowDate: string
): Promise<{ totalFound: number; emailsSent: number; telegramSent: boolean }> {
  const { results } = await db
    .prepare(
      `SELECT id, patient_name, patient_phone, patient_email, service_type, appointment_date, appointment_time
       FROM appointments
       WHERE appointment_date = ? AND status = 'confirmed' AND reminder_sent = 0
       ORDER BY appointment_time ASC`
    )
    .bind(tomorrowDate)
    .all<UpcomingAppointmentRow>();

  if (results.length === 0) {
    return { totalFound: 0, emailsSent: 0, telegramSent: false };
```

```
}

const tasks: Promise<unknown>[] = [];
let telegramSent = false;
let emailsSent = 0;

if (env.TELEGRAM_BOT_TOKEN && env.TELEGRAM_CHAT_ID) {
  const lines = results.map(
    (a, idx) =>
      `${idx + 1}. *${a.appointment_time}* — ${escapeMarkdown(a.patient_name)} (${escapeMarkdown(a.service_type)}) • \
  );

  const digestText = [
    `🗓️ *24-Hour Schedule Digest for Tomorrow (${tomorrowDate})*`,
    `Total confirmed appointments: *${results.length}*`,
    ``,
    ...lines,
  ].join("\n");

  tasks.push(
    fetch(`https://api.telegram.org/bot${env.TELEGRAM_BOT_TOKEN}/sendMessage`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        chat_id: env.TELEGRAM_CHAT_ID,
        text: digestText,
        parse_mode: "Markdown",
      }),
    })
  );
  telegramSent = true;
}

if (env.RESEND_API_KEY) {
  const emailPayloads = results
    .filter((a) => Boolean(a.patient_email))
    .map((a) => ({
      from: "SmileCare Dental <onboarding@resend.dev>",
      to: [a.patient_email!],
      subject: `Reminder: Your Dental Appointment Tomorrow (${a.appointment_date} at ${a.appointment_time})`,
      html: `
        <div style="font-family: sans-serif; max-width: 520px; margin: 0 auto; border: 1px solid #e2e8f0; border-radiu
          <h2 style="color: #0d9488; margin-top: 0;">Friendly 24-Hour Appointment Reminder</h2>
          <p>Hi <strong>${a.patient_name}</strong>,</p>
          <p>This is a reminder of your dental appointment tomorrow at <strong>SmileCare Dental Clinic</strong>:</p>
          <ul style="line-height: 1.8; color: #334155;">
            <li><strong>Service:</strong> ${a.service_type}</li>
            <li><strong>Date:</strong> ${a.appointment_date}</li>
            <li><strong>Time:</strong> ${a.appointment_time}</li>
          </ul>
        </div>
      `,
    }));

  if (emailPayloads.length > 0) {
    tasks.push(
      fetch("https://api.resend.com/emails/batch", {
        method: "POST",
        headers: {
          Authorization: `Bearer ${env.RESEND_API_KEY}`,
          "Content-Type": "application/json",
        },
        body: JSON.stringify(emailPayloads),
      })
    );
    emailsSent = emailPayloads.length;
  }
}

await Promise.allSettled(tasks);

const updateStatements = results.map((a) =>
  db.prepare("UPDATE appointments SET reminder_sent = 1 WHERE id = ?").bind(a.id)
);
```

```
  await db.batch(updateStatements);

  return { totalFound: results.length, emailsSent, telegramSent };
}
```

5. `src/server.ts`

(Includes the `AIChatAgent` Durable Object, a lightweight `/api/chat` endpoint for the zero-build widget, the protected `/api/admin/*` routes, the Telegram webhook, and the daily Cron Trigger)

```
import { AIChatAgent, type OnChatMessageOptions } from "@cloudflare/ai-chat";
import { routeAgentRequest } from "agents";
import { streamText, generateText, tool, convertToModelMessages, stepCountIs, type CoreMessage } from "ai";
import { createWorkersAI } from "workers-ai-provider";
import { z } from "zod";
import { sendBookingNotifications, sendDailyReminders, type NotificationEnv } from "./notifications";

type AppEnv = Env & NotificationEnv & { ADMIN_PASSWORD?: string };

const CLINIC_HOURS = ["09:00", "10:00", "11:00", "13:00", "14:00", "15:00", "16:00"];

function buildSystemPrompt() {
  const today = new Intl.DateTimeFormat("en-CA", {
    timeZone: "Asia/Manila",
    year: "numeric",
    month: "2-digit",
    day: "2-digit",
  }).format(new Date());

  return `You are SmileCare Dental Clinic's friendly virtual receptionist.
Today's date is ${today}.
Clinic Info:
- Hours: Monday–Saturday, 9:00 AM to 5:00 PM (Closed 12:00 PM–1:00 PM for lunch; Closed Sundays).
- Address: 120 Dental Plaza, Suite 4B.
- Services & Pricing:
  * Routine Checkup & Cleaning: $90 (45 mins)
  * Teeth Whitening: $250 (60 mins)
  * Composite Filling: $150–$280 per tooth
  * Root Canal Consultation: $120
  * Emergency Toothache Exam: $100
- Insurance: Delta Dental, Cigna, MetLife, Aetna, and self-pay.

Booking Rules:
1. Always call checkAvailableSlots before promising a time slot.
2. Before calling bookAppointment, confirm you have the patient's full name, phone number, optional email, service type,
3. Keep answers warm, concise, and helpful.`;
}

function createDentalTools(env: AppEnv, ctx: ExecutionContext | DurableObjectState) {
  return {
    checkAvailableSlots: tool({
      description: "Check available dental appointment time slots for a specific date (YYYY-MM-DD).",
      inputSchema: z.object({
        date: z.string().describe("Date in YYYY-MM-DD format"),
      }),
      execute: async ({ date }) => {
        const { results } = await env.DB.prepare(
          "SELECT appointment_time FROM appointments WHERE appointment_date = ? AND status = 'confirmed'"
        )
          .bind(date)
          .all<{ appointment_time: string }>();

        const booked = new Set(results.map((r) => r.appointment_time));
        const available = CLINIC_HOURS.filter((slot) => !booked.has(slot));

        return {
          date,
          availableSlots: available,
          note: available.length === 0 ? "Fully booked on this date." : "Times are in 24-hour HH:MM format.",
        };
      },
    }),

    bookAppointment: tool({
```

```
      description: "Book a dental appointment slot after verifying availability and collecting patient details.",
      inputSchema: z.object({
        patientName: z.string().describe("Full name of the patient"),
        patientPhone: z.string().describe("Contact phone number"),
        patientEmail: z.string().email().optional().describe("Optional patient email address for confirmation"),
        serviceType: z.string().describe("Dental service requested"),
        date: z.string().describe("Appointment date in YYYY-MM-DD format"),
        time: z.string().describe("Appointment time in 24-hour HH:MM format from available slots"),
      }),
      execute: async ({ patientName, patientPhone, patientEmail, serviceType, date, time }) => {
        if (!CLINIC_HOURS.includes(time)) {
          return { success: false, error: `Invalid slot. Choose from: ${CLINIC_HOURS.join(", ")}` };
        }

        const id = crypto.randomUUID();
        const confirmationCode = id.slice(0, 8).toUpperCase();

        try {
          await env.DB.prepare(
            `INSERT INTO appointments (id, patient_name, patient_phone, patient_email, service_type, appointment_date, a
             VALUES (?, ?, ?, ?, ?, ?, ?, 'confirmed', ?)`
          )
            .bind(id, patientName, patientPhone, patientEmail ?? null, serviceType, date, time, new Date().toISOString()
            .run();

          ctx.waitUntil(
            sendBookingNotifications(env, {
              id,
              confirmationCode,
              patientName,
              patientPhone,
              patientEmail,
              serviceType,
              date,
              time,
            })
          );

          return {
            success: true,
            confirmationCode,
            summary: `${serviceType} booked for ${patientName} on ${date} at ${time}.`,
          };
        } catch {
          return {
            success: false,
            error: "That time slot was just taken. Please check available slots again.",
          };
        }
      },
    }),

    cancelAppointment: tool({
      description: "Cancel an existing appointment using the patient's phone number and appointment date.",
      inputSchema: z.object({
        patientPhone: z.string().describe("Patient phone number used during booking"),
        date: z.string().describe("Appointment date in YYYY-MM-DD format"),
      }),
      execute: async ({ patientPhone, date }) => {
        const res = await env.DB.prepare(
          "UPDATE appointments SET status = 'cancelled' WHERE patient_phone = ? AND appointment_date = ? AND status = 'c
        )
          .bind(patientPhone, date)
          .run();

        return res.meta.changes > 0
          ? { success: true, message: "Appointment cancelled." }
          : { success: false, message: "No matching confirmed appointment found." };
      },
    }),
  };
}

export class DentalAgent extends AIChatAgent<AppEnv> {
```

```
  async onChatMessage(_onFinish: unknown, options?: OnChatMessageOptions) {
    const workersai = createWorkersAI({ binding: this.env.AI });

    const result = streamText({
      model: workersai("@cf/meta/llama-3.3-70b-instruct-fp8-fast"),
      system: buildSystemPrompt(),
      messages: await convertToModelMessages(this.messages),
      abortSignal: options?.abortSignal,
      stopWhen: stepCountIs(5),
      tools: createDentalTools(this.env, this.ctx),
    });

    return result.toUIMessageStreamResponse();
  }
}

function verifyAdminPassword(request: Request, env: AppEnv): boolean {
  const authHeader = request.headers.get("Authorization") ?? "";
  const token = authHeader.startsWith("Bearer ") ? authHeader.slice(7) : "";
  const expected = env.ADMIN_PASSWORD ?? "";

  if (!token || !expected) return false;

  const encoder = new TextEncoder();
  const a = encoder.encode(token);
  const b = encoder.encode(expected);

  if (a.byteLength !== b.byteLength) return false;
  return crypto.subtle.timingSafeEqual(a, b);
}

async function handleAdminApi(request: Request, env: AppEnv): Promise<Response> {
  if (!verifyAdminPassword(request, env)) {
    return Response.json({ error: "Unauthorized" }, { status: 401 });
  }

  const url = new URL(request.url);

  if (request.method === "GET" && url.pathname === "/api/admin/appointments") {
    const date = url.searchParams.get("date");
    const status = url.searchParams.get("status");
    const search = url.searchParams.get("search");

    let query = "SELECT * FROM appointments WHERE 1=1";
    const params: string[] = [];

    if (date) {
      query += " AND appointment_date = ?";
      params.push(date);
    }
    if (status && status !== "all") {
      query += " AND status = ?";
      params.push(status);
    }
    if (search) {
      query += " AND (patient_name LIKE ? OR patient_phone LIKE ?)";
      params.push(`%${search}%`, `%${search}%`);
    }

    query += " ORDER BY appointment_date ASC, appointment_time ASC LIMIT 250";

    const { results } = await env.DB.prepare(query).bind(...params).all();
    return Response.json({ appointments: results });
  }

  const match = url.pathname.match(/^\/api\/admin\/appointments\/([a-zA-Z0-9-]+)$/);
  if (request.method === "PATCH" && match) {
    const id = match[1];
    const body = await request.json<{ status?: string }>();
    if (body.status) {
      await env.DB.prepare("UPDATE appointments SET status = ? WHERE id = ?").bind(body.status, id).run();
    }
    return Response.json({ success: true });
  }
```

```
  if (request.method === "DELETE" && match) {
    const id = match[1];
    await env.DB.prepare("DELETE FROM appointments WHERE id = ?").bind(id).run();
    return Response.json({ success: true });
  }

  return Response.json({ error: "Not found" }, { status: 404 });
}

async function handleTelegramWebhook(request: Request, env: AppEnv): Promise<Response> {
  const update = await request.json<{
    callback_query?: { id: string; data?: string };
  }>();

  const cb = update.callback_query;
  if (cb?.data?.startsWith("status:")) {
    const [, appointmentId, newStatus] = cb.data.split(":");
    await env.DB.prepare("UPDATE appointments SET status = ? WHERE id = ?").bind(newStatus, appointmentId).run();

    await fetch(`https://api.telegram.org/bot${env.TELEGRAM_BOT_TOKEN}/answerCallbackQuery`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        callback_query_id: cb.id,
        text: `Appointment marked as ${newStatus}!`,
      }),
    });
  }
  return Response.json({ ok: true });
}

function getTomorrowDateString(timeZone = "Asia/Manila"): string {
  const tomorrow = new Date(Date.now() + 24 * 60 * 60 * 1000);
  return new Intl.DateTimeFormat("en-CA", {
    timeZone,
    year: "numeric",
    month: "2-digit",
    day: "2-digit",
  }).format(tomorrow);
}

export default {
  async fetch(request: Request, env: AppEnv, ctx: ExecutionContext): Promise<Response> {
    const url = new URL(request.url);

    // 1. Lightweight REST chat endpoint for the zero-build HTML widget
    if (url.pathname === "/api/chat" && request.method === "POST") {
      const { messages } = await request.json<{ messages: CoreMessage[] }>();
      const workersai = createWorkersAI({ binding: env.AI });

      const result = await generateText({
        model: workersai("@cf/meta/llama-3.3-70b-instruct-fp8-fast"),
        system: buildSystemPrompt(),
        messages,
        stopWhen: stepCountIs(5),
        tools: createDentalTools(env, ctx),
      });

      return Response.json({ reply: result.text });
    }

    // 2. Admin Portal API
    if (url.pathname.startsWith("/api/admin/")) {
      return handleAdminApi(request, env);
    }

    // 3. Telegram Interactive Button Webhook
    if (url.pathname === "/api/telegram/webhook" && request.method === "POST") {
      return handleTelegramWebhook(request, env);
    }

    // 4. Cloudflare Agents SDK WebSocket routing
    const agentResponse = await routeAgentRequest(request, env);
```

```
    if (agentResponse) return agentResponse;

    // 5. Static HTML pages (public/index.html & public/admin.html)
    return env.ASSETS.fetch(request);
  },

  async scheduled(controller: ScheduledController, env: AppEnv, ctx: ExecutionContext): Promise<void> {
    const tomorrowDate = getTomorrowDateString("Asia/Manila");
    ctx.waitUntil(
      sendDailyReminders(env.DB, env, tomorrowDate).then((summary) => {
        console.log(`[Cron ${controller.cron}] Reminders for ${tomorrowDate}:`, JSON.stringify(summary));
      })
    );
  },
} satisfies ExportedHandler<AppEnv>;
```

6. `public/index.html` (Patient Chat Page + Link to Admin)

(Also save the `public/admin.html` file from earlier right beside this file in `public/`)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>SmileCare Dental Clinic — AI Receptionist</title>
  <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-slate-50 min-h-screen flex flex-col items-center justify-center p-4">
  <div class="w-full max-w-md bg-white rounded-2xl shadow-xl border overflow-hidden flex flex-col h-[560px]">
    <div class="bg-teal-600 text-white px-5 py-4 flex justify-between items-center">
      <div>
        <h1 class="font-bold text-base">SmileCare Dental Assistant</h1>
        <p class="text-xs text-teal-100">Ask questions or book an appointment 24/7</p>
      </div>
      <a href="/admin.html" class="text-xs bg-teal-700 hover:bg-teal-800 px-3 py-1.5 rounded-lg">Admin</a>
    </div>

    <div id="chat-box" class="flex-1 p-4 overflow-y-auto space-y-3 text-sm">
      <div class="bg-slate-100 text-slate-800 p-3 rounded-xl max-w-[85%]">
        Hi! Welcome to SmileCare Dental Clinic. How can I help you today? You can ask about our services, pricing, or bo
      </div>
    </div>

    <form id="chat-form" class="p-3 border-t flex gap-2 bg-white">
      <input
        id="chat-input"
        type="text"
        placeholder="e.g. Do you have openings tomorrow at 10am?"
        required
        class="flex-1 border rounded-xl px-3.5 py-2 text-sm focus:outline-none focus:border-teal-600"
      />
      <button
        id="send-btn"
        type="submit"
        class="bg-teal-600 hover:bg-teal-700 text-white px-4 py-2 rounded-xl text-sm font-medium disabled:opacity-50"
      >
        Send
      </button>
    </form>
  </div>

  <script>
    const chatBox = document.getElementById("chat-box");
    const chatForm = document.getElementById("chat-form");
    const chatInput = document.getElementById("chat-input");
    const sendBtn = document.getElementById("send-btn");

    const messages = [];

    function appendBubble(role, text) {
      const wrapper = document.createElement("div");
      wrapper.className = role === "user" ? "text-right" : "text-left";
```

```
      const bubble = document.createElement("div");
      bubble.className =
        role === "user"
          ? "inline-block bg-teal-600 text-white px-3.5 py-2.5 rounded-xl max-w-[85%] text-left whitespace-pre-wrap"
          : "inline-block bg-slate-100 text-slate-800 px-3.5 py-2.5 rounded-xl max-w-[85%] whitespace-pre-wrap";
      bubble.textContent = text;

      wrapper.appendChild(bubble);
      chatBox.appendChild(wrapper);
      chatBox.scrollTop = chatBox.scrollHeight;
      return bubble;
    }

    chatForm.addEventListener("submit", async (e) => {
      e.preventDefault();
      const text = chatInput.value.trim();
      if (!text) return;

      chatInput.value = "";
      appendBubble("user", text);
      messages.push({ role: "user", content: text });

      sendBtn.disabled = true;
      const thinkingBubble = appendBubble("assistant", "Checking...");

      try {
        const res = await fetch("/api/chat", {
          method: "POST",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({ messages }),
        });
        const data = await res.json();
        thinkingBubble.textContent = data.reply || "Sorry, I couldn't process that.";
        messages.push({ role: "assistant", content: data.reply });
      } catch (err) {
        thinkingBubble.textContent = "Error connecting to assistant. Please try again.";
      } finally {
        sendBtn.disabled = false;
        chatBox.scrollTop = chatBox.scrollHeight;
      }
    });
  </script>
</body>
</html>
```

Step 2: Provision D1 & Run Locally

Run these 4 commands in your terminal:

```
# 1. Log in to your free Cloudflare account (if not already logged in)
npx wrangler login

# 2. Create the D1 database
npx wrangler d1 create dental-appointments-db
```

Copy the `database_id` printed in your terminal and paste it into `wrangler.jsonc` in place of `REPLACE_WITH_YOUR_D1_DATABASE_ID`. Then initialize the database tables and start your server:

```
# 3. Create tables locally and remotely
npx wrangler d1 execute dental-appointments-db --local --file=./schema.sql
npx wrangler d1 execute dental-appointments-db --remote --file=./schema.sql

# 4. Generate TypeScript types and create a local .dev.vars password
npx wrangler types
echo "ADMIN_PASSWORD=admin123" > .dev.vars

# 5. Start the local server (Workers AI will run remotely using your free quota)
npx wrangler dev --test-scheduled
```

Open **`http://localhost:8787`** to test booking an appointment in the chat widget, and **`http://localhost:8787/admin.html`** (password: `admin123`) to see it appear in your admin table.

Step 3: Deploy to Production (`*.workers.dev`)

Once it works locally, push it live to Cloudflare's global network:

```
# Set your production Admin Password
npx wrangler secret put ADMIN_PASSWORD

# Optional: Set Telegram & Resend keys when ready
# npx wrangler secret put TELEGRAM_BOT_TOKEN
# npx wrangler secret put TELEGRAM_CHAT_ID
# npx wrangler secret put RESEND_API_KEY
# npx wrangler secret put CLINIC_NOTIFICATION_EMAIL

# Deploy live
npx wrangler deploy
```

Run `npx wrangler d1 create dental-appointments-db` first and let me know if you hit any errors or want help connecting your Telegram bot token!

User prompt: https://share.gemini.google/oOJymupSxA5m

Response: You shared an `opencode.json` configuration file that registers a remote Model Context Protocol (MCP) server named `comfy-cloud` (`[https://cloud.comfy.org/mcp](https://cloud.comfy.org/mcp)`), along with a Gemini share link (`[https://share.gemini.google/oOJymupSxA5m](https://share.gemini.google/oOJymupSxA5m)`) that I cannot directly read from here.

How would you like to proceed with this? Here are the most likely ways we can use it:

1. **Expose the Dental AI Assistant as a Remote MCP Server (or connect it to one):** The Cloudflare Agents SDK (`agents/mcp`) supports both hosting remote MCP servers on Cloudflare Workers and connecting your `DentalAgent` to external MCP servers like `comfy-cloud`.
2. **Use OpenCode to Build & Deploy the Dental Project:** If you are running OpenCode locally in your `dental-ai-assistant` folder, we can configure `opencode.json` with the Cloudflare MCP server so OpenCode can manage your D1 database and Workers deployments directly.
3. **Something from the Gemini Share Link:** If the shared link contains specific instructions, clinic details, or a workflow you want to integrate, paste the text here and we will wire it right in.
